

Reverse Linked List

Uma questão real de entrevista técnica de Big Tech, resolvida em C e dividida em cinco tópicos de apresentação.

1 · Problema

2 · Simples e Direta

3 · Mais didática

4 · Mais eficiente

5 · Comparativo

Grupo:

Daniela Yukari Udo

Enzo Legnaro Silveira

Lucas Jundi Hikazudani

Luiz Augusto de Oliveira Sellitto

Sarah Gabriele Scandola Frossard

Um problema, cinco tópicos de análise

A apresentação percorre cinco etapas em sequência, cada uma implementada em C e avaliada com Big-O.

1

TÓPICO 1

O problema

Contextualiza o enunciado, as restrições e o que significa inverter a lista.

2

TÓPICO 2

Solução ingênua (simples e direta)

Vetor auxiliar: copia os valores e percorre a lista três vezes.

3

TÓPICO 3

Solução mais didática

Recursão: visualiza a pilha de chamadas desfazendo os links.

4

TÓPICO 4

Solução mais eficiente

Iterativa in-place: troca os ponteiros em uma única passagem.

5

TÓPICO 5

Big-O comparativo

Compara tempo e espaço das soluções e fecha com a conclusão.

Inverter uma lista encadeada simples

Enunciado real de entrevista técnica (LeetCode 206) — ponto de partida da nossa análise.

Enunciado

Dada a head de uma lista encadeada simples, inverta a lista e retorne a nova head.

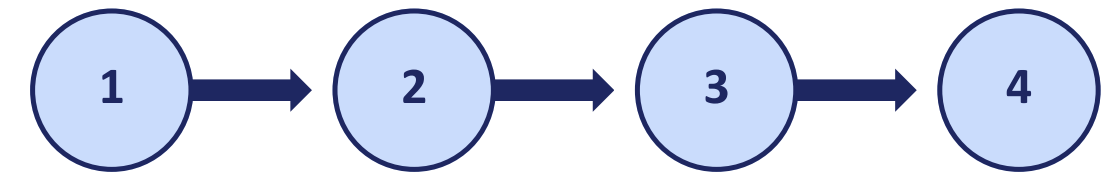
Restrições

- 0 a 5000 nós na lista
- Valores entre -5000 e 5000

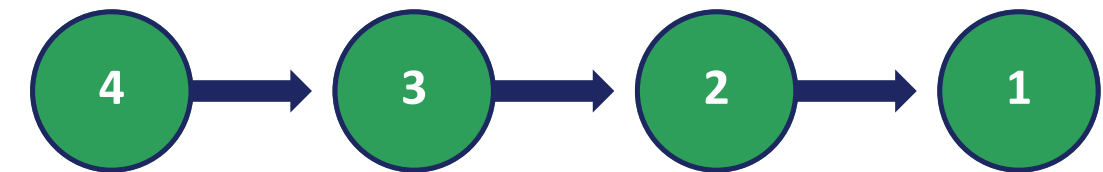
Desafio proposto pelo professor

- Implementar em C
- Avaliar complexidade de tempo e espaço (Big-O)
- Comparar formas iterativa e recursiva

`head = [1, 2, 3, 4]`

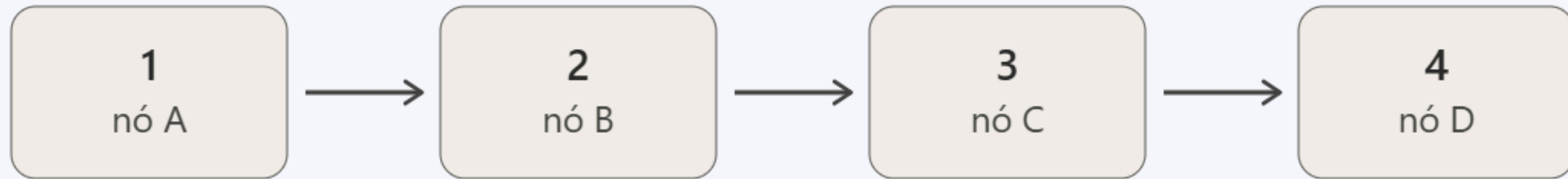


`output = [4, 3, 2, 1]`



Só os ponteiros next mudam de direção — os nós continuam sendo os mesmos na memória.

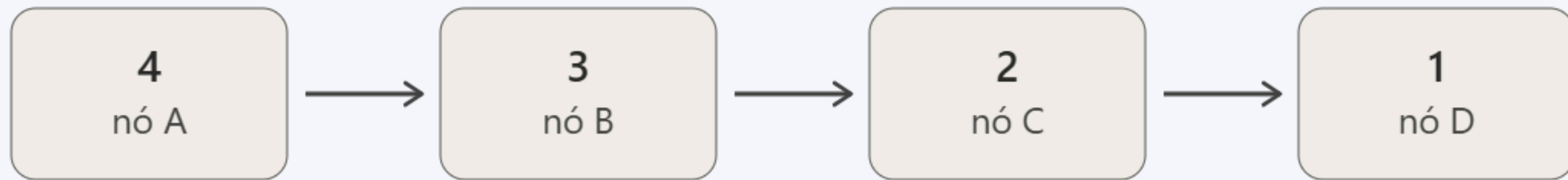
1. lista original — cada nó guarda seu valor



2. copia os valores para o vetor auxiliar



3. regrava lendo o vetor de trás pra frente



os nós A–D não saem do lugar; só o conteúdo de cada um muda



```
/* Inverte uma lista ligada trocando os valores de lugar (os nós ficam parados). */
struct ListNode* reverseList(
    struct ListNode* head) {
    int length = 0;                // quantidade de nós na lista
    struct ListNode* cursor = head; // ponteiro de percurso

    while (cursor) {
        length++;
        cursor = cursor->next;
    }                                // 1ª passada: conta os nós da lista

    int *values = malloc(length * sizeof(int)); // buffer temporário dos valores

    cursor = head;                // volta ao início
    for (int i = 0; i < length; i++) {
        values[i] = cursor->val;    // salva o valor do nó atual
        cursor = cursor->next;
    }                                // 2ª passada: copia a lista para o vetor

    cursor = head;                // volta ao início de novo
    for (int i = length - 1; i ≥ 0; i--) {
        cursor->val = values[i];    // sobrescreve com o valor espelhado
        cursor = cursor->next;
    }                                // 3ª passada: lê o vetor de trás pra
    // frente e regrava na lista (inversão)

    free(values);                // devolve a memória do buffer
    return head;                // mesmo ponteiro, conteúdo invertido
}
```

TEMPO

 $O(n)$

ESPAÇO

 $O(n)$ **ANÁLISE**

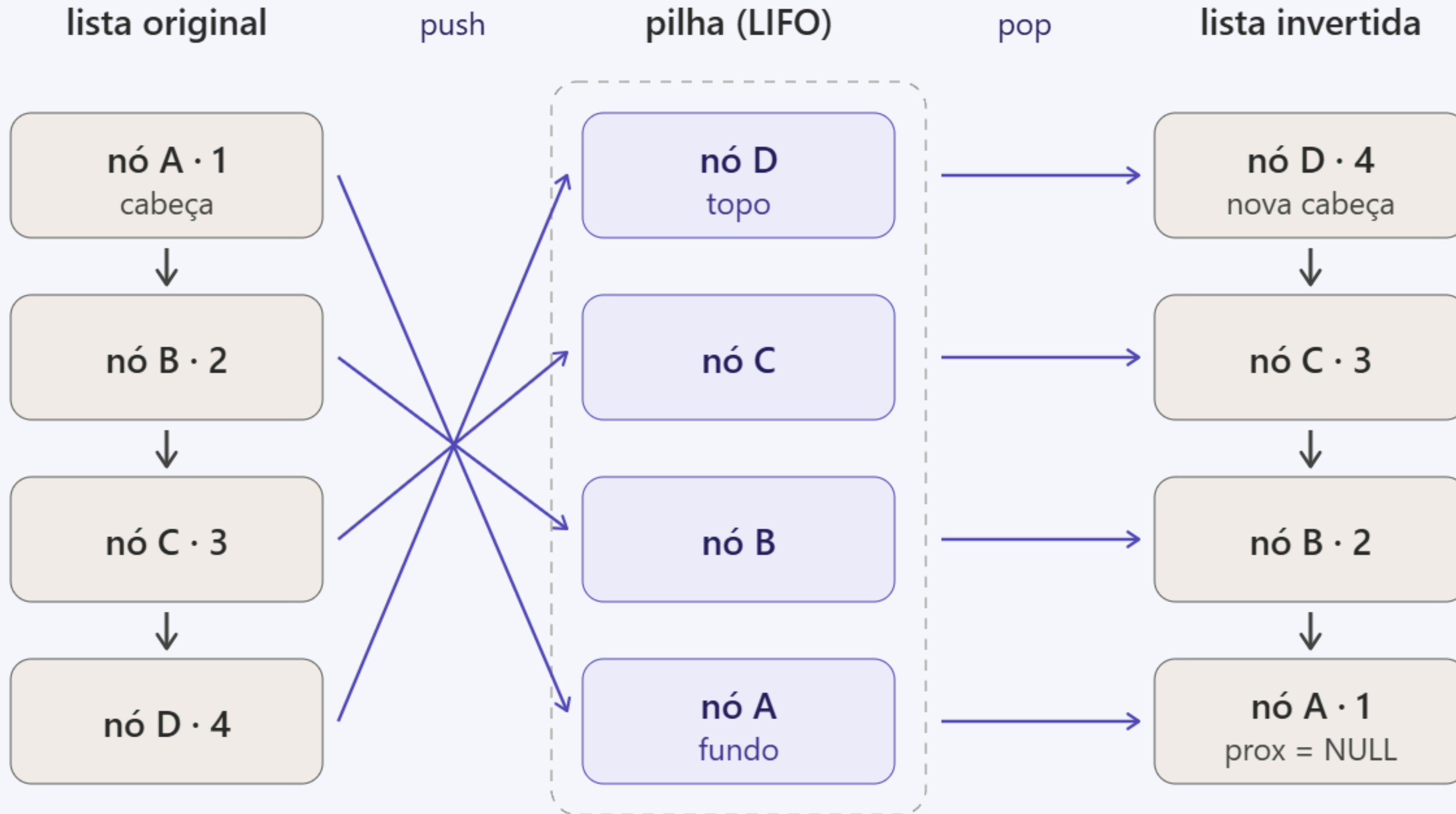
Três iterações pela lista mais malloc/free de um vetor auxiliar. Não inverte os ponteiros, apenas realoca os valores. Resolve o problema, mas ao custo de $O(n)$ de memória extra

LIMITAÇÕES

- O nó fica no lugar, mas o valor dentro dele vira outro.
- Se o nó tivesse outros campos além de val, todos precisariam ser copiados

PONTOS POSITIVOS?

Solução Ingênua 2 (com pilha explícita)



cada nó leva seu valor junto; quem muda é a posição dele na lista

Solução Ingênua 2 (com pilha explícita)

Empilha os nós e religa os ponteiros ao desempilhar



```
/* Pilha LIFO de ponteiros para nós */
int  push(Pilha *p, No *dado); // empilha, 0 se malloc falhar
No  *pop(Pilha *p);           // desempilha, NULL se vazia

/* Inverte a lista religando os ponteiros de cada nó. */
No *inverter(No *cabeca) {
    Pilha p;
    pilha_init(&p);

    for (No *atual = cabeca; atual; atual = atual->prox)
        push(&p, atual);           // 1ª passada: empilha os nós

    No *nova = pop(&p);             // topo = antigo último = nova cabeça
    No *fim  = nova;               // ponta da lista sendo remontada

    while (!pilha_vazia(&p)) {
        fim->prox = pop(&p);        // religa com o nó de baixo
        fim = fim->prox;           // avança a ponta
    }                               // 2ª passada: desempilha ao contrário

    if (fim) fim->prox = NULL;      // antiga cabeça vira o último
    return nova;                   // ponteiro NOVO – o head mudou
}
```

TEMPO

 $O(n)$

ESPAÇO

 $O(n)$

ANÁLISE

Duas iterações, mas agora invertendo os ponteiros. Cada nó mantém seu valor e muda de posição, diferente da solução anterior. Como o head deixa de ser o mesmo nó, a função devolve um ponteiro novo.

O QUE PESA CONTRA

- Um malloc por nó: n alocações contra 1 da solução com vetor
- A pilha só torna explícito o que a recursão faria implicitamente

PONTOS POSITIVOS?

Solução mais didática

Reversão recursiva — ótima para visualizar o conceito

reverseList.c

```
struct ListNode* reverseList(  
    struct ListNode* head) {  
    // Caso base: lista vazia ou 1 nó  
    if (head == NULL ||  
        head->next == NULL) {  
        return head;  
    }  
    // Reverte o restante primeiro  
    struct ListNode* newHead =  
        reverseList(head->next);  
    // Desfaz a recursão: inverte o  
    // link do nó atual  
    head->next->next = head;  
    head->next = NULL;  
    return newHead;  
}
```

TEMPO

$O(n)$

ESPAÇO

$O(n)$

POR QUE ISSO IMPORTA

Cada chamada empilha um quadro até chegar ao último nó, e é 'desfazendo a pilha' que os links são invertidos — ótimo para ensinar recursão. Mas o espaço $O(n)$ da pilha de chamadas pode causar stack overflow em listas grandes (até 5000 nós no enunciado).

Solução mais eficiente

Reversão iterativa in-place — a resposta esperada em entrevista

reverseList.c

```
struct ListNode* reverseList(  
    struct ListNode* head) {  
    struct ListNode* prev = NULL;  
    struct ListNode* curr = head;  
  
    while (curr != NULL) {  
        struct ListNode* next =  
            curr->next;  
        curr->next = prev;  
        prev = curr;  
        curr = next;  
    }  
  
    return prev;  
}
```

TEMPO

$O(n)$

ESPAÇO

$O(1)$

POR QUE ISSO IMPORTA

Uma única passagem pela lista, trocando apenas três ponteiros por nó (prev, curr, next). Nenhuma alocação extra e nenhum uso de pilha adicional — é a solução que qualquer entrevistador de Big Tech espera ver como resposta final.

As três soluções, lado a lado

Todas resolvem o problema — o que muda é o custo em tempo e espaço de cada abordagem.

Abordagem	Tempo	Espaço	Observação
Vetor auxiliar (tópico 2)	$O(n)$	$O(n)$	3 passagens na lista + malloc/free
Recursiva (tópico 3)	$O(n)$	$O(n)$ — pilha	Risco de stack overflow em listas grandes
Iterativa in-place (tópico 4)	$O(n)$	$O(1)$	Solução recomendada em entrevistas

Conclusão prática: as três resolvem o problema corretamente e em tempo $O(n)$ — a diferença está no espaço extra usado e na clareza pedagógica de cada implementação.

RECAPITULANDO

Eficiência não é só velocidade — é o par (tempo, espaço)

- 1 Todas as soluções são $O(n)$ em tempo — o que as diferencia é o espaço extra usado.
- 2 A solução recursiva ensina o conceito com clareza, mas paga o preço da pilha de chamadas.
- 3 A iterativa in-place é a esperada em processos seletivos: $O(n)$ tempo, $O(1)$ espaço.
- 4 Analisar Big-O antes de escrever código evita escolher a solução 'que só funciona'.

Obrigado! Perguntas?